

Technical Communication - Assignment 6

UTKARSH KUMAR 23114101

March 2026

1. (a) Algorithm in $\LaTeX$ with Filled Blanks

Algorithm 1 Dijkstra's Algorithm

For each vertex $v \in V$:
 $\text{dist}[v] \leftarrow \infty$, $\text{visited}[v] \leftarrow \text{False}$
Then set:
 $\text{dist}[s] \leftarrow 0$
Repeat for $|V|$ iterations:
 • choose the unvisited vertex u with minimum $\text{dist}[u]$
 • if $\text{dist}[u] = \infty$, stop
 • set $\text{visited}[u] \leftarrow \text{True}$
 • for each neighbor v of u in alphabetical order:
 – if $\text{visited}[v] = \text{False}$ and $\text{dist}[u] + w(u, v) < \text{dist}[v]$
 – then set $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$
Return dist

(b) Execution Table in $\LaTeX$ with Filled Entries

Step	Finalized Node	Distance Vector (A, B, C, D, E, F)
0	–	$(0, \infty, \infty, \infty, \infty, \infty)$
1	A	$(0, 4, 2, \infty, \infty, \infty)$
2	C	$(0, 3, 2, 10, 12, \infty)$
3	B	$(0, 3, 2, 8, 12, \infty)$
4	D	$(0, 3, 2, 8, 10, 14)$
5	E	$(0, 3, 2, 8, 10, 13)$
6	F	$(0, 3, 2, 8, 10, 13)$

Table 1: Distance Table

(c) Final Result in $\LaTeX$

Shortest distance from A to F = 13

Shortest path from A to F = A $\rightarrow$ C $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E $\rightarrow$ F

(d) Precision Statement in $\LaTeX$

In Dijkstra's Algorithm, the relaxation step checks whether a new discovered path to a vertex is shorter than the previously known shortest path. If the condition $\text{dist}[u] + w(u, v) < \text{dist}[v]$ holds, the algorithm updates $\text{dist}[v]$ to $\text{dist}[u] + w(u, v)$.

2. In the following, we show how to use the `minipage` environment to combine text and logic circuit diagrams as shown in fig.1.

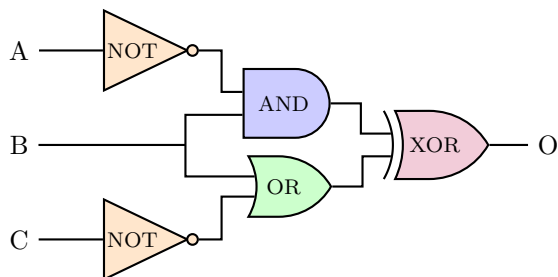


Figure 1: Combinational Logic Circuit

Digital Logic Design

The logic circuit shown in the image is a combinational system that processes three binary inputs (A, B, and C) through three stages of logic to determine the final output O. In the first stage, inputs A and C are passed through NOT gates to create their inverted complements, $\neg A$ and $\neg C$. Next, an AND gate combines $\neg A$ with input B, while an OR gate combines input B with $\neg C$. Finally, these two intermediate results are fed into an XOR (Exclusive OR) gate, which produces a "True" output only if the signals from the AND and OR stages are different. The overall behavior can be summarized by the Boolean expression $O = (\neg A \wedge B) \oplus (B \vee \neg C)$.